



JavaFX Bean Property的前生——

JavaBean规范及实现

北京理工大学计算机学院
金旭亮

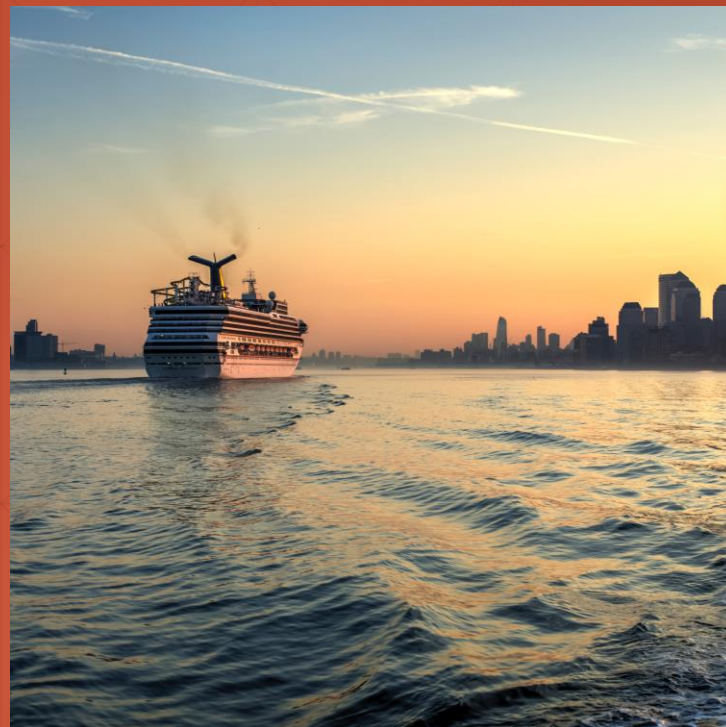
什么是JavaBean?



- ✓ JavaBean可以看成是一种编程约定，按照这种约定编写Java类，开发者之间就易于协作，并且代码也易于重用，并且有一些开发工具（比如IntelliJ这样的IDE）能识别JavaBean所定义的各种属性和方法。
- ✓ 比如，开发Java桌面应用时，所有的系统内置UI控件（比如JavaFX控件）都是JavaBean，修改这些控件的属性，会直接影响到控件的状态和外观。
- ✓ 简单地说，你可以把JavaBean看成是按照大家约定方式编写的Java类。

JavaBean的官方文档：

<http://docs.oracle.com/javase/tutorial/javabeans/index.html>



学会编写JavaBean

定义JavaBean的属性

JavaBean属性 = 私有字段 + get方法 + set方法

```
2 usages
private int intField;
no usages
public int getIntField() {
    return intField;
}
no usages
public void setIntField(int intField) {
    this.intField = intField;
}
```

```
// boolean类型的字段, 其Get方法格式为: isFieldName()
2 usages
private boolean running;
no usages
public boolean isRunning() {
    return running;
}
no usages
public void setRunning(boolean running) {
    this.running = running;
}
```



只读属性没有set方法，只写属性没有get方法。

支持索引的属性 (Indexed Properties)

// 支持索引的属性

4 usages

```
private int[] values = new int[] { 1, 2, 3, 4 };
```

no usages

```
public int[] getValues() {  
    return values;  
}
```

no usages

```
public void setValues(int[] values) {  
    this.values = values;  
}
```

no usages

```
public int getValues(int index) {  
    return values[index];  
}
```

no usages

```
public void setValues(int index, int value) {  
    values[index] = value;  
}
```

这种属性，既可以直接设置或访问其整个数组，也可以单独访问其成员。

数据依赖性



绑定

```
double sum = x + y;
```

依赖

A red bracket connects the text '绑定' (Binding) to the expression 'x + y' in the code. A red dashed box encloses 'x + y', and the text '依赖' (Dependency) is placed below it.

当 x, y 数值变化时， sum 的值也会变化，我们称 sum “依赖”于 x, y ，这种数据之间的依赖性，可以使用“绑定 (Bind)”来体现。

被绑定的属性 (Bound Properties)



- ✓ 为一个JavaBean定义好一个可绑定的属性之后，就可以给它添加外部的监听器（即Listener），当属性值一更改，所有监听器对象就会得到通知。
- ✓ java.beans包中包容了两个类型：**PropertyChangeEvent**和**PropertyChangeListener**。可以调用addPropertyChangeListener和removePropertyListener这两个方法来为可绑定JavaBean属性添加或移除监听器对象，当属性值有变化时，JavaBean会触发一个**PropertyChangeEvent**事件，并通知所有注册的**PropertyChangeListener**。
- ✓ java.beans包中还有一个PropertyChangeSupport类，使用它可以很方便地管理监听器对象，并且在需要时能随时触发事件。

Bound Properties实例

```
// 支持绑定的属性
3 usages
private int bindableField = 0;
// 用于触发事件
3 usages
private PropertyChangeSupport listeners =
    new PropertyChangeSupport( sourceBean: this);
// 绑定属性的get/set方法
no usages
public int getBindableField() { return bindableField; }
2 usages
public void setBindableField(int bindableField) {
    int oldBindableField = this.bindableField;
    this.bindableField = bindableField;
    // 触发“属性值更改”事件
    listeners.firePropertyChange( propertyName: "bindableField",
        oldBindableField, bindableField);
}
// 添加和移除属性改变事件监听器
1 usage
public void addPropertyChangeListener(PropertyChangeListener listener) {
    listeners.addPropertyChangeListener(listener);
}
no usages
public void removePropertyChangeListener(PropertyChangeListener listener) {
    listeners.removePropertyChangeListener(listener);
}
```

对于这种属性，只要它的值一更改，就会触发一个事件，注册的事件监听器对象会得到通知。

事件参数中同时包含老值和新值

绑定属性的测试代码



```
private static void testPropertyChangeEvent() {  
    MyBean bean = new MyBean();  
    //添加事件监听器  
    bean.addPropertyChangeListener(new PropertyChangeListener() {  
        @Override  
        public void propertyChange(PropertyChangeEvent evt) {  
            String info = String.format("属性名:%s,老值:%s,新值:%s",  
                evt.getPropertyName(), evt.getOldValue().toString(),  
                evt.getNewValue().toString());  
            System.out.println(info);  
        }  
    });  
    //触发两次"PropertyChange"事件  
    bean.setBindableField(100);  
    bean.setBindableField(200);  
}
```

输出:

```
Run  [icon] [icon] [icon] [icon]  
↑ "C:\Program Files\Java\jdk-17.0.3.1\bin\java.exe"  
↓ 属性名:bindableField,老值:0,新值:100  
↔ 属性名:bindableField,老值:100,新值:200  
≡  
☐  
☑ Process finished with exit code 0
```

约束属性 (Constrained Properties)



约束属性是可绑定属性的一种。可以为约束属性添加一个或多个有“投票权”的监听器 (veto listener)，当属性值将要改变时，由这些监听器对象投票决定是否这个值应该改变。



如果监听器对象不赞同这个值应该改变，抛出PropertyVetoException。



在实际开发中，通常使用java.beans包中的VetoableChangeSupport类来实现约束属性。

约束属性的示例

```
// 约束属性
3 usages
private int score = 0;
3 usages
private VetoableChangeSupport vetoListeners =
    new VetoableChangeSupport( sourceBean: this);
//约束属性的get/set方法
no usages
public int getScore() { return score; }
1 usage
public void setScore(int score) throws PropertyVetoException {
    int oldValue = this.score;
    //通知“投票监听器”进行“投票”，不同意者抛出PropertyVetoException异常
    vetoListeners.fireVetoableChange( propertyName: "score", oldValue, score);
    this.score = score;
}
//添加与删除“投票监听器”
1 usage
public void addVetoableChangeListener(VetoableChangeListener listener) {
    vetoListeners.addVetoableChangeListener(listener);
}
no usages
public void removeVetoableChangeListener(VetoableChangeListener listener) {
    vetoListeners.removeVetoableChangeListener(listener);
}
```

同一个属性，可以同时注册 `PropertyChangeListener` 和 `VetoableChangeListener`，在触发事件时，注意要把后者放到前面，仅当值的改变“通过投票”后，才真正改变值并触发 `PropertyChange` 事件。

约束属性的测试代码

```
private static void testConstrainedProperties() {  
    // 添加一个“成绩有效性”监听器  
    var scoreValidateListener = new VetoableChangeListener() {  
        5 usages  
        @Override  
        public void vetoableChange(PropertyChangeEvent evt)  
            throws PropertyVetoException {  
            int newValue = Integer.parseInt(evt.getNewValue().toString());  
            String propertyName = evt.getPropertyName();  
            if (propertyName.equals("score") && (newValue < 0 || newValue > 100)) {  
                throw new PropertyVetoException("有效的成绩在0分与100分之间", evt);  
            }  
        }  
    };  
};
```

只要有任何一个投票者投出了“反对票”，对属性值的修改就会失败。

可以根据实际场景，添加实现了不同判断逻辑的“投票表决者”。

```
//挂接JavaBean与监听器  
MyBean bean = new MyBean();  
bean.addVetoableChangeListener(scoreValidateListener);  
try {  
    //随机生成一个整数  
    int scoreValue = new Random().nextInt( bound: 200);  
    System.out.println("要设定的成绩为" + scoreValue);  
    //只要生成的数值不在[0,100]之间，将抛出异常  
    bean.setScore(scoreValue);  
} catch (PropertyVetoException e) {  
    System.out.println(e.getMessage());  
}  
}
```

测试结果



成绩修改成功：



```
Run  [icon] [icon] [icon] [icon]
↑      "C:\Program Files\Java\jdk-17.0.3.1\bin\java.exe"
↓      要设定的成绩为61
≡
≡↓
Print  Process finished with exit code 0
```

成绩修改失败：



```
Run  [icon] [icon] [icon] [icon]
↑      "C:\Program Files\Java\jdk-17.0.3.1\bin\java.exe"
↓      要设定的成绩为106
≡      有效的成绩在0分与100分之间
≡↓
Print  Process finished with exit code 0
Trash
```

小结



JavaBean作为一种普遍遵循的Java编程规范，应该在实际开发中遵照执行。



JavaBean规范所定义各种事件和相关类型，在Java EE中得到了应用。



JavaFX针对桌面应用的特点，在JavaBean规范的基础上扩充和设计了自己的支持数据绑定的属性，称为“JavaFX Bean Property”，这部分内容，放在JavaFX部分进行介绍。